

National University of Technology

Machine Learning Lab- CS3504

LAB Submitted to:

Lec.Hajra Ahmad

Submitted By:

Muhammad Abdullah

Reg. No: **F22607010**
Batch: **Fall AI 2022**
Lab : **10**

Q.NO.1:

1. **Real Time CLustring.**

Answer:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from collections import deque

# Constants for K-Means
K = 3  # Number of clusters
MAX_ITER = 10  # Maximum number of iterations for centroid update
DISTANCE_THRESHOLD = 2.0  # Outlier detection threshold
CONCEPT_DRIFT_THRESHOLD = 0.1  # Threshold to detect concept drift

# Function to calculate Euclidean distance
def euclidean_distance(point1, point2):
    return np.sqrt(np.sum((point1 - point2) ** 2))

# Class to manage the K-Means clustering
class RealTimeKMeans:
    def __init__(self, k, max_iter=10):
        self.k = k
        self.max_iter = max_iter
        self.centroids = None
        self.cluster_stats = {}  # To track cluster count, mean, and
variance

    def initialize_centroids(self, data_points):
        """Initialize centroids by selecting random points from data"""
        random_indices = np.random.choice(len(data_points), self.k,
replace=False)
        self.centroids = data_points[random_indices]

    def assign_points_to_centroids(self, data_points):
        """Assign each point to the nearest centroid"""
        cluster_assignments = []
        for point in data_points:
            distances = [euclidean_distance(point, centroid) for centroid
in self.centroids]
```

```
            cluster_assignments.append(np.argmin(distances))
        return cluster_assignments

    def update_centroids(self, data_points, cluster_assignments):
        """Update centroids by calculating the mean of assigned points"""
        new_centroids = np.zeros_like(self.centroids)
        for i in range(self.k):
            assigned_points = [data_points[j] for j in
range(len(data_points)) if cluster_assignments[j] == i]
            if assigned_points:
                new_centroids[i] = np.mean(assigned_points, axis=0)
        return new_centroids

    def update_cluster_stats(self, data_points, cluster_assignments):
        """Update cluster statistics: count, mean, and variance"""
        for i in range(self.k):
            assigned_points = [data_points[j] for j in
range(len(data_points)) if cluster_assignments[j] == i]
            if assigned_points:
                count = len(assigned_points)
                mean = np.mean(assigned_points, axis=0)
                variance = np.var(assigned_points, axis=0)
                self.cluster_stats[i] = {'count': count, 'mean': mean,
'variance': variance}

    def detect_outliers(self, data_points, cluster_assignments):
        """Identify outliers based on the distance threshold"""
        outliers = []
        for i, point in enumerate(data_points):
            cluster_id = cluster_assignments[i]
            centroid = self.centroids[cluster_id]
            distance = euclidean_distance(point, centroid)
            if distance > DISTANCE_THRESHOLD:
                outliers.append(i)
        return outliers

    def detect_concept_drift(self, old_centroids):
        """Detect concept drift by comparing centroids"""
        if self.centroids is None or old_centroids is None:
            return False
        drift_detected = False
        for i in range(self.k):
```

```
            if euclidean_distance(self.centroids[i], old_centroids[i]) >
CONCEPT_DRIFT_THRESHOLD:
                drift_detected = True
        return drift_detected

    def fit(self, data_points):
        """Fit the model by initializing centroids and updating
iteratively"""
        self.initialize_centroids(data_points)
        old_centroids = None

        for _ in range(self.max_iter):
            cluster_assignments =
self.assign_points_to_centroids(data_points)
            self.update_cluster_stats(data_points, cluster_assignments)
            new_centroids = self.update_centroids(data_points,
cluster_assignments)

            # Check for concept drift
            if self.detect_concept_drift(old_centroids):
                print("Concept drift detected!")
                self.reinitialize_centroids(data_points)

            # Update centroids
            old_centroids = self.centroids
            self.centroids = new_centroids

    def reinitialize_centroids(self, data_points):
        """Reinitialize centroids in case of concept drift"""
        self.initialize_centroids(data_points)

    def plot_clusters(self, data_points, cluster_assignments):
        """Visualize the clusters in real-time"""
        plt.clf()
        for i in range(self.k):
            cluster_points = [data_points[j] for j in
range(len(data_points)) if cluster_assignments[j] == i]
            cluster_points = np.array(cluster_points)
            plt.scatter(cluster_points[:, 0], cluster_points[:, 1],
label=f"Cluster {i}")
        plt.scatter(self.centroids[:, 0], self.centroids[:, 1],
color='black', marker='x', s=100, label="Centroids")
        plt.legend()
```

```
        plt.draw()
        plt.pause(0.1)

# Function to simulate the data stream
def generate_stream(num_points=50):
    """Simulate incoming 2D data points."""
    return np.random.randn(num_points, 2) * 2

# Real-time clustering simulation
def simulate_real_time_clustering():
    # Creating the KMeans model
    model = RealTimeKMeans(k=3)

    # Simulate incoming stream of data points
    data_stream = deque(maxlen=200)  # to simulate a continuous stream of
data points

    plt.ion()  # Interactive mode to allow real-time plotting
    for _ in range(100):  # Simulate 100 iterations of the data stream
        # Generate new data points
        new_data_points = generate_stream(5)  # 5 new points at each time
step
        data_stream.extend(new_data_points)

        # Perform clustering on the current data stream
        model.fit(np.array(data_stream))
        cluster_assignments =
model.assign_points_to_centroids(np.array(data_stream))

        # Plot the current clustering state
        model.plot_clusters(np.array(data_stream), cluster_assignments)

        # Detecting outliers
        outliers = model.detect_outliers(np.array(data_stream),
cluster_assignments)
        print(f"Outliers detected at indices: {outliers}")

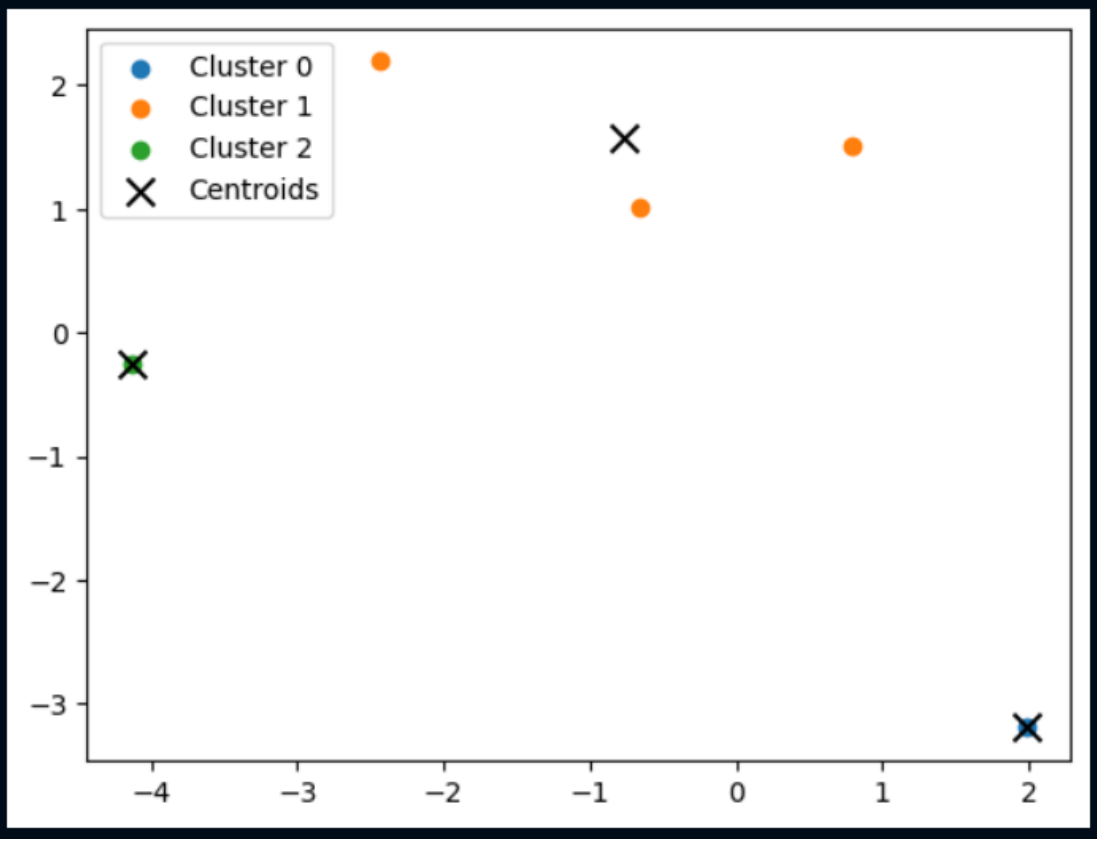
        plt.pause(0.1)

    plt.ioff()  # Turn off interactive plotting
    plt.show()

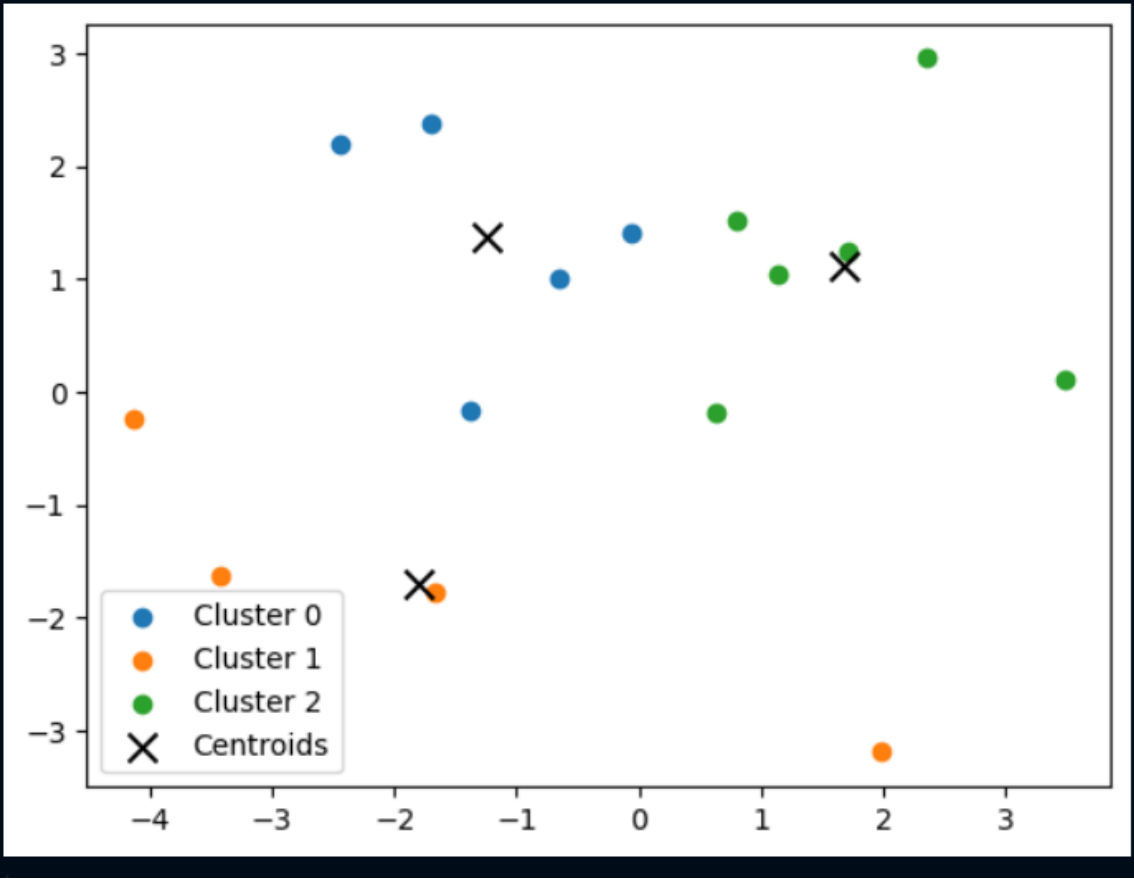
simulate_real_time_clustering()
```

Output:

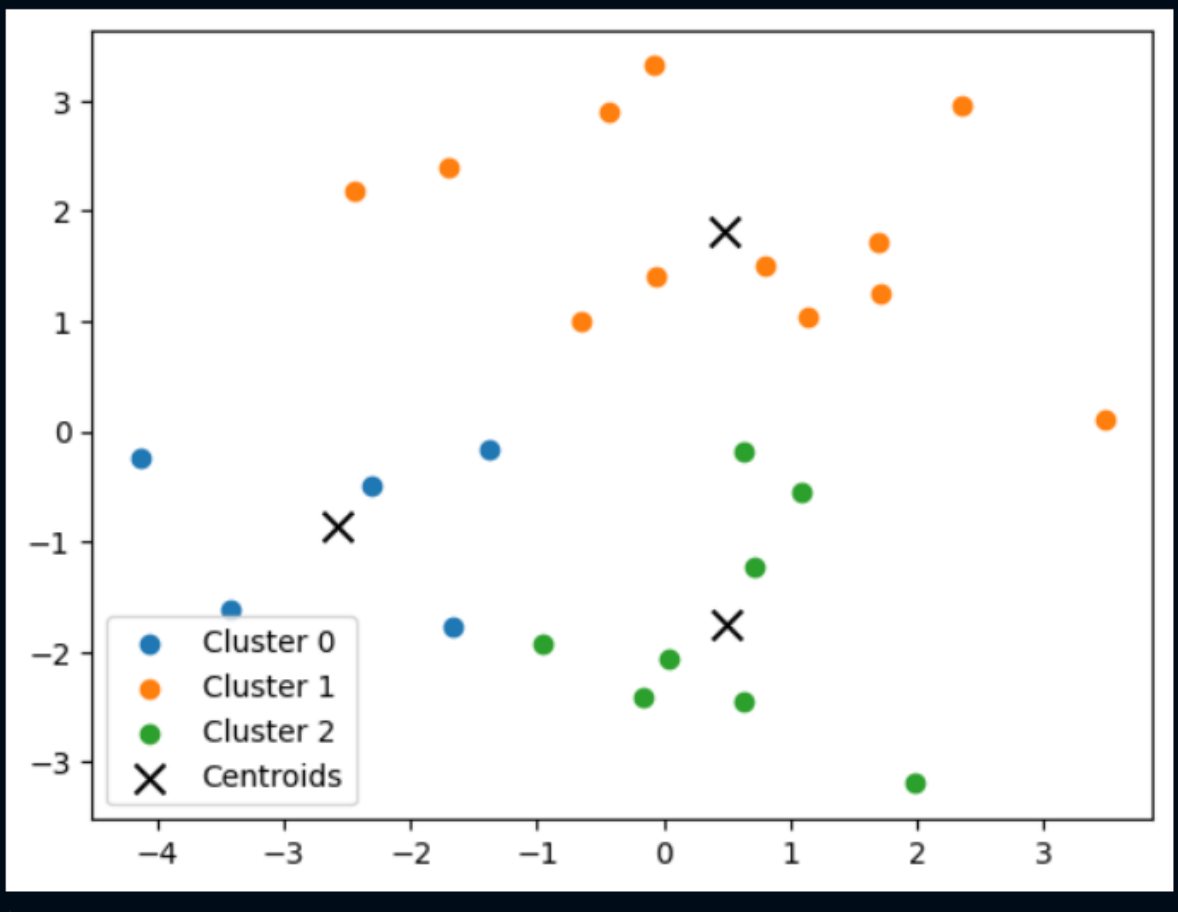
```
Concept drift detected!
Concept drift detected!
Concept drift detected!
Concept drift detected!
Concept drift detected!
Concept drift detected!
Concept drift detected!
Concept drift detected!
Concept drift detected!
```



```
Outliers detected at indices: [2, 4]
Concept drift detected!
Concept drift detected!
Concept drift detected!
Concept drift detected!
Concept drift detected!
Concept drift detected!
Concept drift detected!
Concept drift detected!
Concept drift detected!
```



```
Outliers detected at indices: [3, 4, 7, 8, 12, 19]
Concept drift detected!
Concept drift detected!
Concept drift detected!
Concept drift detected!
Concept drift detected!
Concept drift detected!
Concept drift detected!
Concept drift detected!
Concept drift detected!
```



And so on…….